PRACTICAL NO. 7

AIM: A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint) The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. (Sample data for reference is given below)

Process	Allocation Printer	Allocation ROM	Allocation Hard Disk	Allocation RAM	Max Printer	Max ROM	Max Hard Disk	Max RAM
Google Drive	0	0	1	4	0	6	5	6
Firefox	0	6	3	2	0	6	5	2
Word Processor	0	0	1	2	0	0	1	2
Excel	1	0	0	0	1	7	5	0
PowerPoint	1	3	5	4	2	3	5	6

Available Resources			
Printer	**ROM**	**Hard Disk**	**RAM**
1	6	2	0

Write a program to:

- Calculate current need of resources by each process Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

CODE:

```
#include <stdio.h>

int main()
{
 int i,j,k;
 int n = 5; // processes
 int m = 4; // resources
```

```
int alloc[5][4] = {
   {0,0,1,4},
   {0,6,3,2},
   {0,0,1,2},
   {1,0,0,0},
   {1,3,5,4}
};

int max[5][4] = {
   {0,6,5,6},
   {0,6,5,2},
   {0,0,1,2},
   {1,7,5,0},
   {2,3,5,6}
};

int avail[4] = {1,6,2,0};

int need[5][4];
int finish[5]={0};
int safe[5];

printf("\nALLOCATION TABLE\n");
for(i=0;i<n;i++){
   for(j=0;j<m;j++){
      printf("%d ",alloc[i][j]);
   }
   printf("\n");
}

printf("\nMAX TABLE\n");
for(i=0;i<n;i++){
   for(j=0;j<m;j++){
      printf("%d ",max[i][j]);
   }
   printf("\n");
}

for(i=0;i<n;i++){
   for(j=0;j<m;j++){
      need[i][j] = max[i][j] - alloc[i][j];
   }
}
```

```
printf("\nNEED TABLE\n");
for(i=0;i<n;i++){
   for(j=0;j<m;j++){
      printf("%d ",need[i][j]);
   }
   printf("\n");
}

printf("\nAVAILABLE RESOURCES\n");
for(i=0;i<m;i++){
   printf("%d ",avail[i]);
}

int count=0;

while(count<n){
   for(i=0;i<n;i++){
      if(finish[i]==0){
         int flag=0;

         for(j=0;j<m;j++){
            if(need[i][j] > avail[j]){
               flag=1;
               break;
            }
         }

         if(flag==0){
            for(k=0;k<m;k++){
               avail[k]+=alloc[i][k];
            }

            safe[count]=i;
            finish[i]=1;
            count++;
         }
      }
   }
}

printf("\n\nSAFE SEQUENCE\n");
```

```
 for(i=0;i<n;i++){
    printf("P%d ",safe[i]);
 }

 printf("\n");

 return 0;

}
```

OUTPUT:

shreyansh123@DESKTOP-1UJ64CI: ~

```
shreyansh123@DESKTOP-1UJ64CI:~$ nano banker.c
shreyansh123@DESKTOP-1UJ64CI:~$ gcc banker.c -o banker
shreyansh123@DESKTOP-1UJ64CI:~$ ./ banker
-bash: ./: Is a directory
shreyansh123@DESKTOP-1UJ64CI:~$ ./banker

ALLOCATION TABLE
0 0 1 4
0 6 3 2
0 0 1 2
1 0 0 0
1 3 5 4

MAX TABLE
0 6 5 6
0 6 5 2
0 0 1 2
1 7 5 0
2 3 5 6

NEED TABLE
0 6 4 2
0 0 2 0
0 0 0 0
0 7 5 0
1 0 0 2

AVAILABLE RESOURCES
1 6 2 0

SAFE SEQUENCE
P1 P2 P3 P4 P0
shreyansh123@DESKTOP-1UJ64CI:~$
```